

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров
семейства Intel – разработка ассемблерной вставки»

Выполнил:
Студент 4-го курса
Группы АС-66
Савинец М. Д.
Проверил:
Кулик А.Д.

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

Ход работы

Вариант 6

Нечётные символы заменить на символ +.

Позиции символов отсчитываются с единицы. Поэтому изменяются элементы строки с индексами 0, 2, 4 и так далее. Например, строка abcdef преобразуется в +b+d+f. Чётные позиции и длина строки сохраняются.

Среда выполнения

Работа выполнена на Mac с процессором Apple Silicon, архитектура ARM64. Использованы Apple Clang 21.0.0, стандарт C++17 и оптимизация -O2. Выполнение на Mac согласовано с преподавателем. Измеренные результаты относятся к ARM64; они не являются результатами запуска Intel-версии.

Алгоритм обработки

Созданы две отдельные функции: `replace_cpp` на C++ и `replace_asm` с ассемблерной вставкой. На вход передаются адрес массива символов и его длина. Обе функции заменяют каждый второй символ, начиная с первого. Для пустой строки записи в память не выполняются.

Ввод UTF-8 преобразуется в `std::u32string`. Один элемент `char32_t` занимает 4 байта и хранит одну кодовую точку Unicode, поэтому обработка сохраняет русские буквы. Переход к следующей нечётной позиции требует смещения на 8 байт. Составные графические символы могут включать несколько кодовых точек.

В ARM64-вставке `cbz` проверяет нулевой счётчик, `str` записывает код 43 символа +, `add` изменяет адрес, `subs` уменьшает счётчик и обновляет флаги, `b.ne` повторяет цикл. Количество записей равно $n / 2 + n \% 2$. Временная сложность обеих функций — $O(n)$, дополнительная память внутри функций — $O(1)$.

Текст программы

Ниже приведены исходные файлы программы. В transform.cpp условная компиляция выбирает ARM64-вставку при сборке на этом Mac; ветки Intel сохранены для совместимости. Преобразование кодировки, проверки и измерение времени находятся вне функций обработки. В листинге эмодзи записаны эквивалентными escape-последовательностями Unicode.

transform.h

```
#pragma once
#include <cstdint>
// Positions are counted from one; n is the number of Unicode code points.
// Both functions modify the buffer in place and do not touch its terminator.
void replace_cpp(char32_t* text, std::size_t n);
void replace_asm(char32_t* text, std::size_t n);
const char* assembly_backend();
```

transform.cpp

```
#include "transform.h"
static_assert(sizeof(char32_t) == 4, "A character must occupy four bytes");
#ifdef _MSC_VER
#define NOINLINE __declspec(noinline)
#else
#define NOINLINE __attribute__((noinline))
#endif
NOINLINE void replace_cpp(char32_t* text, std::size_t n) {
    // Positions 1, 3, 5 correspond to indices 0, 2, 4.
    for (std::size_t i = 0; i < n; i += 2)
        text[i] = U '+';
}
NOINLINE void replace_asm(char32_t* text, std::size_t n) {
    // Avoid n + 1: that expression could overflow for the largest size_t.
    std::size_t count = n / 2 + n % 2;
#ifdef _MSC_VER && defined(_M_IX86)
    // Main course implementation: Intel x86, MS Visual Studio, Win32.
    // EAX = current address, ECX = remaining replacements, EDX = U '+'.
    __asm {
        mov eax, text
        mov ecx, count
        test ecx, ecx
        jz finished
        mov edx, 43
    next_character:
        mov dword ptr [eax], edx
        add eax, 8
        dec ecx
        jnz next_character
    finished:
    }
#elseif defined(__aarch64__) && defined(__clang__)
    // Mac Apple Silicon adaptation. This is ARM64, not Intel assembly.
    asm volatile(
        "cbz %[count], 2f\n"
        "mov w9, #43\n"
        "1:\n"
        "str w9, [%[text]]\n"
        "add %[text], %[text], #8\n"
        "subs %[count], %[count], #1\n"
        "b.ne 1b\n"
        "2:\n"
        : [text] "+&r"(text), [count] "+&r"(count)
        :
    );
#endif
}
```

```

        : "x9", "cc", "memory");
#elif defined(__x86_64__) && (defined(__clang__) || defined(__GNUC__))
    // Optional Intel x86-64 build on macOS/Linux (GNU assembler syntax).
    asm volatile(
        "testq %[count], %[count]\n"
        "jz 2f\n"
        "1:\n"
        "movl $43, (%[text])\n"
        "addq $8, %[text]\n"
        "decq %[count]\n"
        "jnz 1b\n"
        "2:\n"
        : [text] "+&r"(text), [count] "+&r"(count)
        :
        : "cc", "memory");
#else
#error Unsupported target. Use MSVC Win32 (x86) or Clang on macOS.
#endif
}

const char* assembly_backend() {
#ifdef _MSC_VER && defined(_M_IX86)
    return "Intel x86 / MSVC inline __asm";
#elif defined(__aarch64__)
    return "ARM64 / Apple Silicon (Mac adaptation)";
#else
    return "Intel x86-64 / GNU-style inline asm";
#endif
}

utf8.h

#pragma once
#include <stdint>
#include <stdexcept>
#include <string>

// Encoding conversion is outside the functions being compared and timed.
inline std::u32string decode_utf8(const std::string& input) {
    std::u32string result;
    for (std::size_t i = 0; i < input.size(); i++) {
        auto c = static_cast<unsigned char>(input[i]);
        std::uint32_t cp;
        unsigned extra;
        std::uint32_t minimum;
        if (c < 0x80) { cp = c; extra = 0; minimum = 0; }
        else if (c >= 0xc2 && c <= 0xdf) {
            cp = c & 0x1f; extra = 1; minimum = 0x80;
        } else if (c >= 0xe0 && c <= 0xef) {
            cp = c & 0x0f; extra = 2; minimum = 0x800;
        } else if (c >= 0xf0 && c <= 0xf4) {
            cp = c & 7; extra = 3; minimum = 0x10000;
        } else { throw std::runtime_error("Invalid UTF-8 input"); }
        for (unsigned j = 0; j < extra; ++j) {
            if (i == input.size())
                throw std::runtime_error("Truncated UTF-8 input");
            c = static_cast<unsigned char>(input[i]);
            if ((c & 0xc0) != 0x80)
                throw std::runtime_error("Invalid UTF-8 continuation");
            cp = (cp << 6) | (c & 0x3f);
        }
        if (cp < minimum || cp > 0x10ffff || (cp >= 0xd800 && cp <= 0xdfff))
            throw std::runtime_error("Invalid Unicode code point");
        result.push_back(static_cast<char32_t>(cp));
    }
    return result;
}

inline std::string encode_utf8(const std::u32string& input) {

```

```

std::string result;
for (std::uint32_t cp : input) {
    if (cp > 0x10ffff || (cp >= 0xd800 && cp <= 0xdfff))
        throw std::runtime_error("Invalid Unicode code point");
    if (cp < 0x80) result += static_cast<char>(cp);
    else if (cp < 0x800) {
        result += static_cast<char>(0xc0 | (cp >> 6));
        result += static_cast<char>(0x80 | (cp & 0x3f));
    } else if (cp < 0x10000) {
        result += static_cast<char>(0xe0 | (cp >> 12));
        result += static_cast<char>(0x80 | ((cp >> 6) & 0x3f));
        result += static_cast<char>(0x80 | (cp & 0x3f));
    } else {
        result += static_cast<char>(0xf0 | (cp >> 18));
        result += static_cast<char>(0x80 | ((cp >> 12) & 0x3f));
        result += static_cast<char>(0x80 | ((cp >> 6) & 0x3f));
        result += static_cast<char>(0x80 | (cp & 0x3f));
    }
}
return result;
}

```

main.cpp

```

#include "transform.h"
#include "utf8.h"
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>
#ifdef _WIN32
#define NOMINMAX
#include <windows.h>
#endif
using Transform = void (*)(char32_t*, std::size_t);
void require(bool ok, const char* message) {
    if (!ok) throw std::runtime_error(message);
}
void show(const std::string& input) {
    const auto original = decode_utf8(input);
    auto cpp = original, assembly = original;
    replace_cpp(cpp.data(), cpp.size());
    replace_asm(assembly.data(), assembly.size());
    require(cpp == assembly, "Implementations disagree");
    std::cout << "Input: [" << input << "]\n"
               << "C++:  [" << encode_utf8(cpp) << "]\n"
               << "ASM:   [" << encode_utf8(assembly) << "]\n"
               << "Match: YES\n\n";
}
void tests() {
    const std::vector<std::pair<std::u32string, std::u32string>> cases = {
        {U"", U""}, {U"a", U"+"}, {U"ab", U"+b"},
        {U"abc", U"+b+"}, {U"abcdef", U"+b+d+f"},
        {U"abcde", U"+b+d+"}, {U"a b c", U"+ + "+"},
        {U"++++", U"++++"}, {U"Привет", U"+p+B+T"},
        {U"A\U0001F642E\U0001F680", U"+\U0001F642+\U0001F680"}, {U"\t\n\t",
U"+\n+"}
    };
    for (const auto& item : cases) {
        require(decode_utf8(encode_utf8(item.first)) == item.first,
               "UTF-8 round trip failed");
        for (Transform fn : {replace_cpp, replace_asm}) {
            std::vector<char32_t> buffer(item.first.size() + 2, U'@');

```

```

        std::copy(item.first.begin(), item.first.end(), buffer.begin() +
1);
        fn(buffer.data() + 1, item.first.size());
        require(buffer.front() == U'@' && buffer.back() == U'@',
            "Function wrote outside its buffer");
        require(std::equal(item.second.begin(), item.second.end(),
            buffer.begin() + 1), "Known-answer test
failed");
    }
}
// Independently check each position, including zero and long lengths.
std::mt19937 rng(606);
for (std::size_t n = 0; n <= 1024; ++n) {
    std::u32string input(n, U' ');
    for (auto& c : input) c = static_cast<char32_t>(32 + rng() % 95);
    for (Transform fn : {replace_cpp, replace_asm}) {
        auto output = input;
        fn(output.data(), output.size());
        require(output.c_str()[n] == U'\0', "Terminator modified");
        for (std::size_t i = 0; i < n; ++i)
            require(output[i] == (i % 2 == 0 ? U'+' : input[i]),
                "Position invariant failed");
    }
}
const std::vector<std::string> bad = {
    std::string("\xc0\xaf", 2), std::string("\x80", 1),
    std::string("\xe2\x82", 2), std::string("\xed\xa0\x80", 3),
    std::string("\xf4\x90\x80\x80", 4)
};
for (const auto& s : bad) {
    bool rejected = false;
    try { (void)decode_utf8(s); }
    catch (const std::runtime_error&) { rejected = true; }
    require(rejected, "Invalid UTF-8 accepted");
}
std::cout << "PASS: 11 known-answer cases (both functions, boundary
guards)\n"
    << "PASS: 1025 lengths (both functions, position invariants)\n"
    << "PASS: UTF-8 round trips and 5 invalid inputs\n";
}
// Volatile dispatch plus separate translation units keep calls observable.
// No console output, copying, allocation or encoding inside the timed
region.
double measure(Transform fn, const std::u32string& input, std::size_t
repeats) {
    auto data = input;
    fn(data.data(), data.size()); // warm up
    data = input;
    Transform volatile dispatch = fn;
    const auto start = std::chrono::steady_clock::now();
    for (std::size_t i = 0; i < repeats; ++i)
        dispatch(data.data(), data.size());
    const auto end = std::chrono::steady_clock::now();
    for (std::size_t i = 0; i < data.size(); ++i)
        require(data[i] == (i % 2 == 0 ? U'+' : input[i]),
            "Benchmark result incorrect");
    return std::chrono::duration<double, std::nano>(end - start).count()
        / static_cast<double>(repeats);
}
void benchmark() {
    std::cout << "Benchmark: 7 rounds, median, warm buffer, repeated in-place
calls\n"
        << "Ratio = C++ / ASM; greater than 1 means ASM is faster\n"
        <<
"length,repeats,cpp_ns_per_call,asm_ns_per_call,cpp_div_asm\n";

```

```

for (std::size_t n : {16u, 256u, 4096u, 65536u}) {
    std::u32string input(n, U'a');
    for (std::size_t i = 0; i < n; ++i) input[i] = U'a' + i % 26;
    const auto repeats = std::max<std::size_t>(1000,
        std::min<std::size_t>(2000000, 160000000 / n));
    std::vector<double> cpp, assembly;
    for (int round = 0; round < 7; ++round) {
        // Alternate order to reduce systematic order bias.
        if (round % 2 == 0) {
            cpp.push_back(measure(replace_cpp, input, repeats));
            assembly.push_back(measure(replace_asm, input, repeats));
        } else {
            assembly.push_back(measure(replace_asm, input, repeats));
            cpp.push_back(measure(replace_cpp, input, repeats));
        }
    }
    std::sort(cpp.begin(), cpp.end());
    std::sort(assembly.begin(), assembly.end());
    std::cout << n << ', ' << repeats << ', ' << std::fixed
        << std::setprecision(3) << cpp[3] << ', ' << assembly[3]
        << ', ' << cpp[3] / assembly[3] << '\n';
}
}

int main(int argc, char** argv) {
#ifdef _WIN32
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
#endif
    try {
        std::cout << "Lab 1 | Variant 6 | Replace positions 1, 3, 5, ... with
+\\n"
            << "Backend: " << assembly_backend() << '\\n';
#ifdef __clang__
        std::cout << "Compiler: Clang " << __clang_version__ << '\\n';
#elif defined(_MSC_VER)
        std::cout << "Compiler: MSVC " << _MSC_VER << '\\n';
#endif
        const std::string mode = argc > 1 ? argv[1] : "";
        if (mode == "--test") tests();
        else if (mode == "--bench") benchmark();
        else if (mode == "--demo") {
            for (const auto& s : {"abcdef", "abcde", "Привет", "a b c", ""})
                show(s);
        } else if (mode.empty()) {
            std::cout << "Enter a UTF-8 string (spaces allowed):\\n";
            std::string input;
            if (!std::getline(std::cin, input))
                throw std::runtime_error("No input received");
            show(input);
        } else {
            std::cerr << "Usage: lab1 [--demo | --test | --bench]\\n";
            return 2;
        }
    } catch (const std::exception& error) {
        std::cerr << "Error: " << error.what() << '\\n';
        return 1;
    }
}

```

Результаты выполнения

Функции дали одинаковые результаты на строках чётной и нечётной длины, русском тексте, строке с пробелами и пустой строке. На рисунке 1 показан сохранённый вывод запуска программы в режиме --demo.

Проверка обработки строк

Лабораторная работа №1 · Вариант 6 · Сохранённый вывод запуска на Mac ARM64

```
Lab 1 | Variant 6 | Replace positions 1, 3, 5, ... with +
Backend: ARM64 / Apple Silicon (Mac adaptation)
Compiler: Clang 21.0.0 (clang-2100.3.34.2)
Input: [abcdef]
C++:   [+b+d+f]
ASM:   [+b+d+f]
Match: YES

Input: [abcde]
C++:   [+b+d+]
ASM:   [+b+d+]
Match: YES

Input: [Привет]
C++:   [+p+v+t]
ASM:   [+p+v+t]
Match: YES

Input: [a b c]
C++:   [+ + +]
ASM:   [+ + +]
Match: YES

Input: []
C++:   []
ASM:   []
Match: YES
```

Рисунок 1 — Скриншот просмотра сохранённого вывода программы

Автоматические проверки пройдены: 11 примеров с известным ответом и контролем границ, 1025 длин от 0 до 1024 с проверкой каждой позиции, преобразование UTF-8 и отклонение пяти некорректных входов. Дополнительный запуск с AddressSanitizer и UndefinedBehaviorSanitizer завершился без ошибок. Inline assembly этими средствами не инструментируется; записи по краям буфера проверялись контрольными символами.

Сравнение быстродействия

Время измерялось монотонными часами `std::chrono::steady_clock`. Для каждой длины выполнены семь раундов, порядок запуска функций чередовался. В таблице приведена медиана времени одного вызова в наносекундах. Выделение памяти, копирование, преобразование UTF-8 и вывод не входят в измеряемый интервал.

В каждом раунде многократно обрабатывался один прогретый буфер. После первого вызова нечётные позиции уже содержали плюсы, но следующие вызовы снова выполняли записи. Обе функции работали в одинаковых условиях. Вызовы не встраивались; использовался `volatile`-указатель на функцию.

Длина	Повторов	C++ нс/вызов	ASM нс/вызов	ASM / C++
16	2 000 000	1,892	3,416	1,81
256	625 000	14,635	34,389	2,35
4096	39 062	232,729	473,699	2,04
65536	2441	4628,414	7416,667	1,60

Таблица 1 — Время обработки и отношение времени ASM к C++

Сравнение быстродействия

Лабораторная работа №1 · Вариант 6 · Сохранённый вывод запуска на Mac ARM64

```
Lab 1 | Variant 6 | Replace positions 1, 3, 5, ... with +
Backend: ARM64 / Apple Silicon (Mac adaptation)
Compiler: Clang 21.0.0 (clang-2100.3.34.2)
Benchmark: 7 rounds, median, warm buffer, repeated in-place calls
Ratio = C++ / ASM; greater than 1 means ASM is faster
length, repeats, cpp_ns_per_call, asm_ns_per_call, cpp_div_asm
16, 2000000, 1.892, 3.416, 0.554
256, 625000, 14.635, 34.389, 0.426
4096, 39062, 232.729, 473.699, 0.491
65536, 2441, 4628.414, 7416.667, 0.624
```

Рисунок 2 — Скриншот просмотра сохранённых измерений

Вывод

Реализовал обработку строки по варианту 6 двумя способами и проверил совпадение результатов. Закрепил работу с адресами памяти, регистрами, счётчиком цикла и условными переходами в ассемблерной вставке. В выполненных измерениях оптимизированная функция C++ работала примерно в 1,6–2,35 раза быстрее ARM64-вставки. Ручная запись алгоритма на ассемблере сама по себе не гарантирует ускорения. Полученные значения относятся к указанной конфигурации и обработке прогретого буфера.